Computer Networks

5. Network Layer: Control Plane 1

Network Layer: Control Plane

Goal:

- Understand principles behind network control plane:
  - traditional routing algorithms
  - SDN controllers
- implementation in the Internet:
  - OSPF, BGP
  - OpenFlow
  - Internet Control Message Protocol

Network Layer - Control Plane: Overview

- Introduction
- Routing protocols
  - link state
  - distance vector
- Intra-ISP routing: OSPF
- Routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol

네트워크 계층 기능

데이터 평면 (*Data plane*)

- forwarding: 라우터의 입력 포트에서 적절한 라우터 출력 포트로 패킷 이동

제어 평면 (*Control plane*)

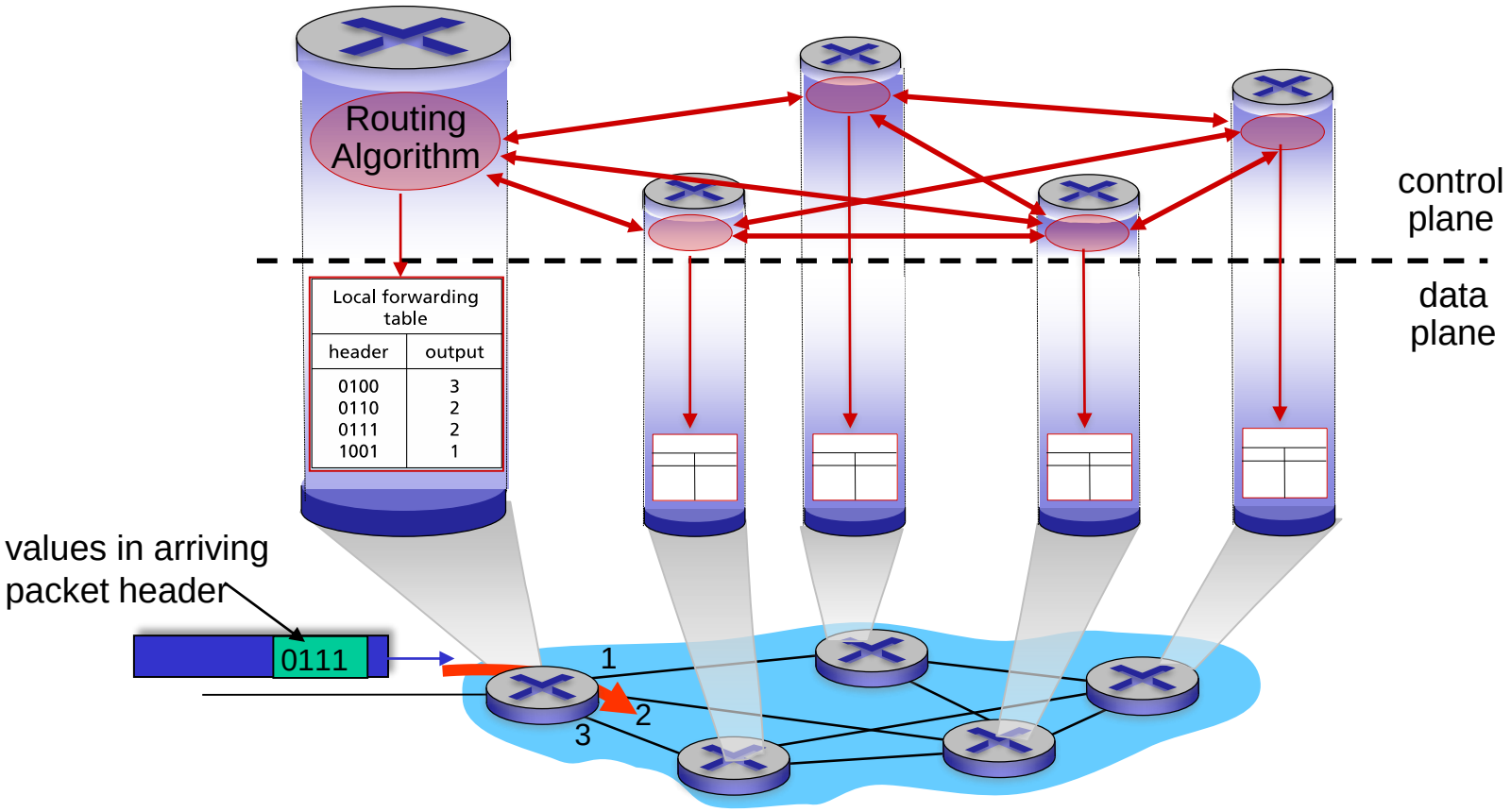
- routing: 출발지부터 목적지까지 패킷이 이동할 경로를 결정

네트워크 제어 평면을 구성하는 두 가지 접근 방식:

- 라우터별 제어 (전통적인 방식)
- 논리적 중앙 집중형 제어 (SDN 방식)

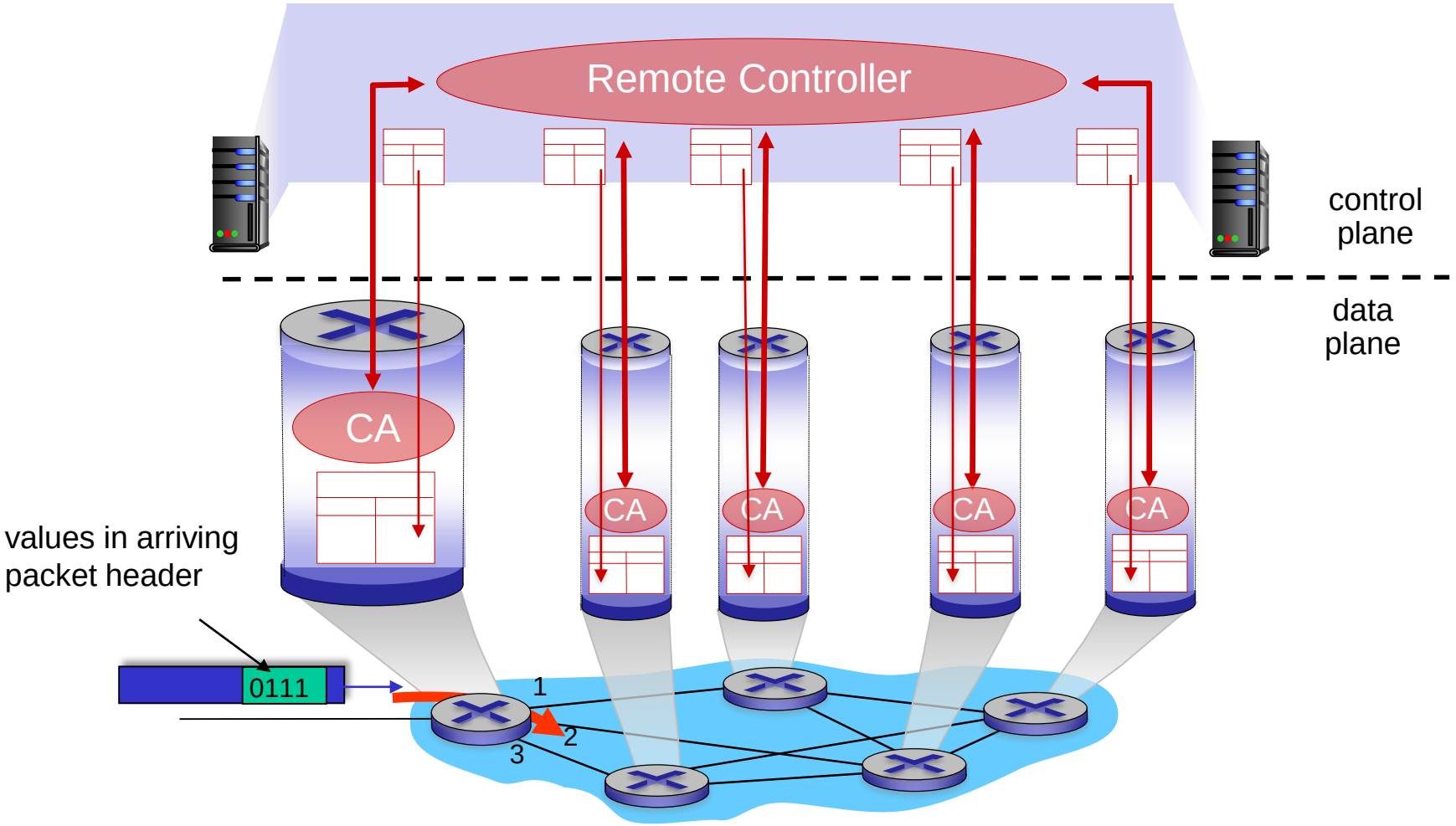
라우터별 제어

각각의 모든 라우터의 개별 라우팅 알고리즘 구성 요소가 제어 평면에서 상호 작용함



논리적 중앙 집중형 제어 (SDN 방식)

원격 컨트롤러가 각 패킷 스위치의 제어 에이전트 (control agent, CA)와 상호작용하여 플로우 테이블을 구성 및 관리



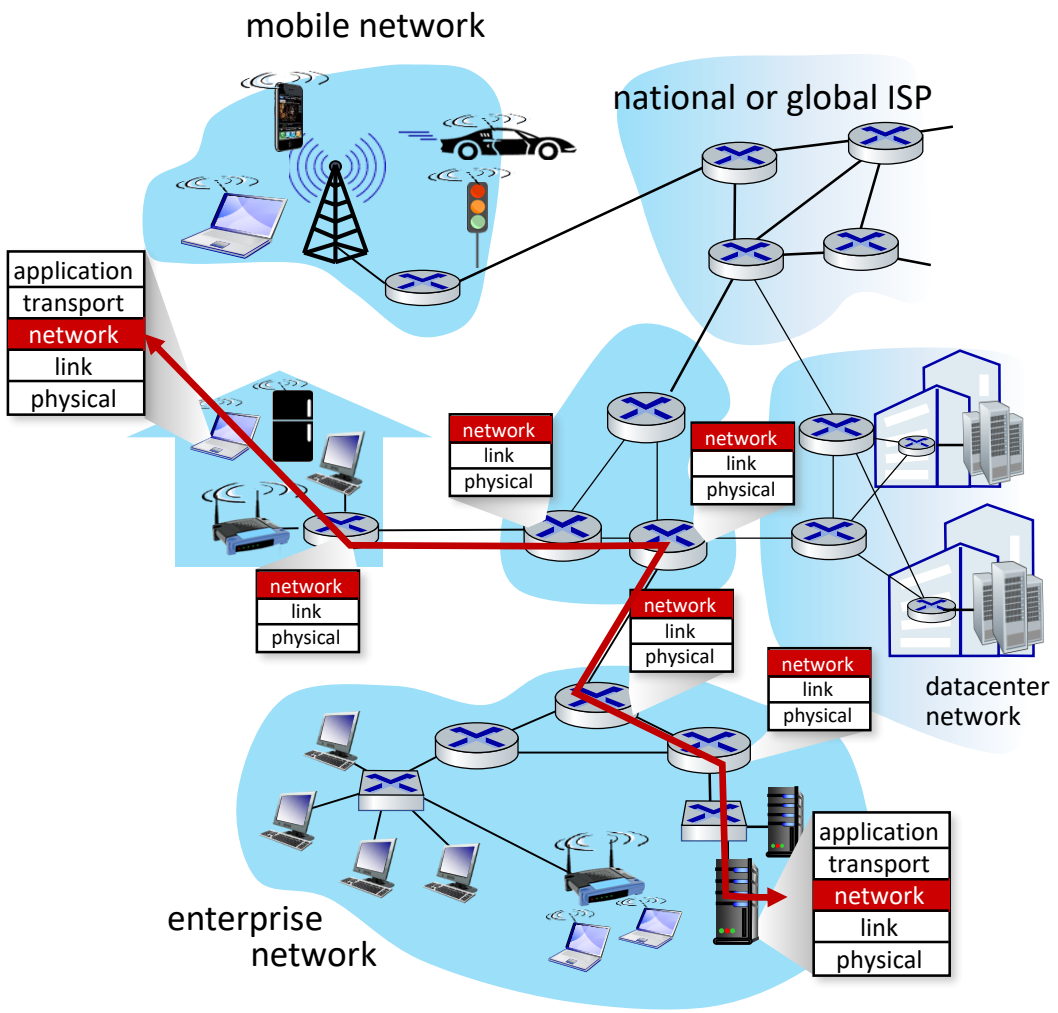
Network Layer - Control Plane: Overview

- Introduction
- Routing protocols
  - link state
  - distance vector
- Intra-ISP routing: OSPF
- Routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol

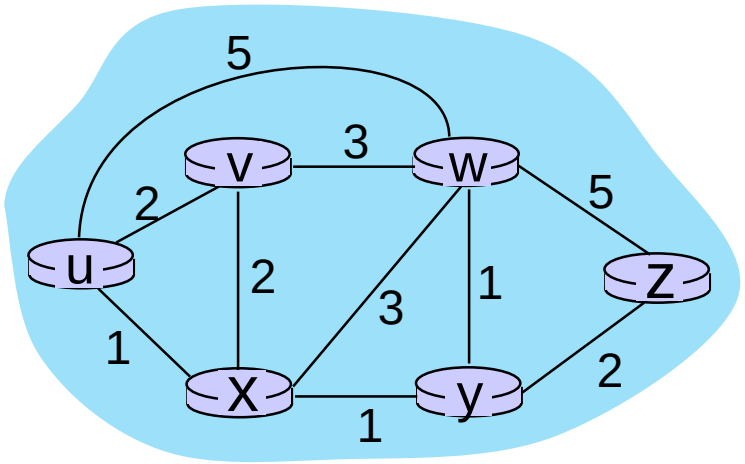
Routing protocols

라우팅 알고리즘의 목표: 출발지 호스트부터 목적지 호스트까지 라우터의 네트워크를 통과하는 가장 '좋은 (good)' 경로 (route/path)를 결정

- 경로: 출발지 호스트에서 목적지 호스트까지 패킷이 통과하는 라우터의 순서
- '좋은': '최소 비용', '가장 빠른', '혼잡이 가장 적은'
  - 일반적으로 최소 비용 경로 (least-cost path)



Graph abstraction: link costs



$c_{a,b}$: A와 B를 연결하는 직접 (direct) 링크의 비용

- [예], $c_{w,z} = 5$, $c_{u,z} = \infty$
- 네트워크 관리자가 '링크의 비용'을 정의: 항상 1이거나, 대역폭과 반비례하거나 또는 혼잡과 비례할 수 있음

graph: $G = (N,E)$

N: 노드 (라우터)의 집합 = { u, v, w, x, y, z }

E: 에지 (링크)의 집합 ={ $(u,v), (u,x), (u,w), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z)$ }

cost of path $(x_1, x_2, x_3, \ldots, x_{p-1}, x_p) = c_{x1,x2} + c_{x1,x2} + \ldots + c_{p-1,p}$

라우팅 알고리즘 (Routing algorithm): 최소 비용 경로를 찾는 알고리즘

라우팅 알고리즘 분류

- 중앙 집중형 (centralized/global): 모든 라우터는 완전한 망구성방식 (topology), 링크 비용 정보를 가짐
  - 링크 상태 (link state, LS) 알고리즘

How fast do routes change?

- 정적 (static): routes change slowly over time
  - routes are changed manually by administrator

- 분산형 (decentralized): 직접 연결된 이웃 라우터와 정보 교환으로 링크 비용을 반복적인 계산
  - 라우터는 초기에 연결된 이웃에 대한 링크 비용만 알고 있음
  - 거리 벡터 (distance vector, DV) 알고리즘

- 동적 (dynamic): routes change more quickly
  - periodic updates or in response to link cost changes

global or decentralized information?

Network Layer - Control Plane: Overview

- Introduction
- Routing protocols
  - link state
  - distance vector
- Intra-ISP routing: OSPF
- Routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol

링크 상태 라우팅 알고리즘

- 링크 상태 광고 (Link-State Advertisement, LSA) 메시지
  - 이웃 라우터 정보
  - 이웃 라우터와 직접 연결된 링크의 상태 및 비용 정보

operation:

- ① 라우터는 다음의 경우에 LSA 메시지 생성
  - 이웃 라우터 변경 (이웃 라우터 추가 또는 삭제)
  - 링크 연결 상태 및 비용 변경
  - 또는 주기적으로
- ② LSA 메세지는 네트워크의 모든 라우터로 플러딩 (flooding)
- ③ 다른 라우터들이 전송한 LSA 메시지들을 기반으로, 각 라우터는 네트워크 전체 토폴로지 (topology)를 구성
- ④ Dijkstra의 알고리즘을 통해 두 라우터 사이의 최소 비용 경로 계산

Dijkstra's link-state routing algorithm

- 중앙 집중형 (centralized): 네트워크 토폴로지와 링크 비용 정보가 모든 노드에 공유됨
  - LSA 메시지를 플러딩 (flooding)하여 정보를 전달하므로, 모든 노드는 동일한 정보를 가짐
- 하나의 노드 (출발지)에서 다른 모든 노드까지 최소 비용 경로를 계산
  - 해당 노드에 대한 포워딩 테이블을 제공
- 반복적 (iterative): *k*번 반복 후, *k*개의 목적지에 대한 최소 비용 경로를 알게 됨

표기법

- $c_{x,y}$: <u>direct</u> link cost from node *x* to *y*; if not direct neighbors = ∞
- *D(v)*: 현재 추정된 출발지에서 목적지 v까지의 최소 비용 경로의 비용
- *p(v)*: 출발지에서 목적지 v까지 현재 최소 비용 경로에서 v의 직전 노드
- *N'*: 최소 비용 경로가 명확히 알려진 노드 집합

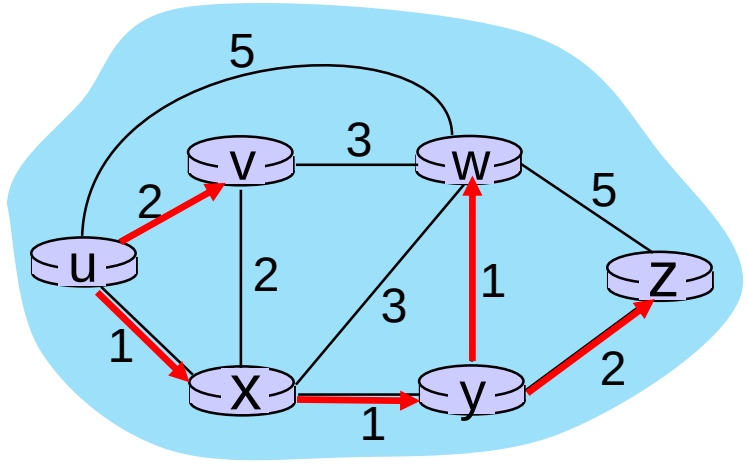
Dijkstra's link-state routing algorithm

```
Initialization:
  N' = {u}                          /* compute least cost path from u to all other nodes */
  for all nodes v
    if v adjacent to u              /* u initially knows direct-path-cost only to direct neighbors */
        then D(v) = c_{u,v}         /* but may not be minimum cost! */
    else D(v) = ∞

Loop
    N'에 포함되지 않은 노드 w 중에서 D(w)가 최소인 노드를 찾아,
    w를 N' 에 추가
    w에 인접하고 N' 에 속하지 않은 모든 노드 v에 대해 D(v)를 갱신:
        D(v) = min ( D(v), D(w) + c_{w,v} )
    /* new least-path-cost to v is either old least-cost-path to v or known
    least-cost-path to w plus direct-cost from w to v */
until all nodes in N'
```

Dijkstra's algorithm: an example

Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

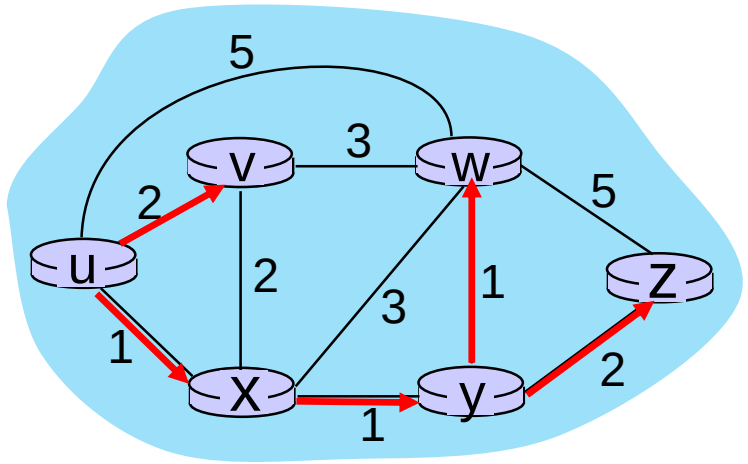


Initialization (step 0): For all a: if *a* adjacent to then $D(a) = c_{u,a}$

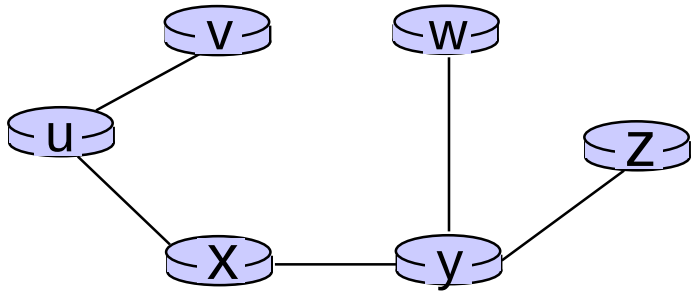
find *a* not in *N'* such that *D(a)* is a minimum
add *a* to *N'*
update *D(b)* for all *b* adjacent to *a* and not in *N'* :

$$D(b) = \min\,(\,D(b),\, D(a) + c_{a,b}\,)$$

Dijkstra's algorithm: an example



노드 *u*로부터의 최소 비용 경로:



노드 *u*의 포워딩 테이블

destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

route from *u* to *v* directly

route from u to all other destinations via *x*

- 직전 노드를 추적하여 최소 비용 경로 트리를 구성
  - 경로는 직전 노드의 역순
- 같은 비용이 존재할 경우, 임의로 노드를 선택

Dijkstra's algorithm: discussion

계산 복잡도:

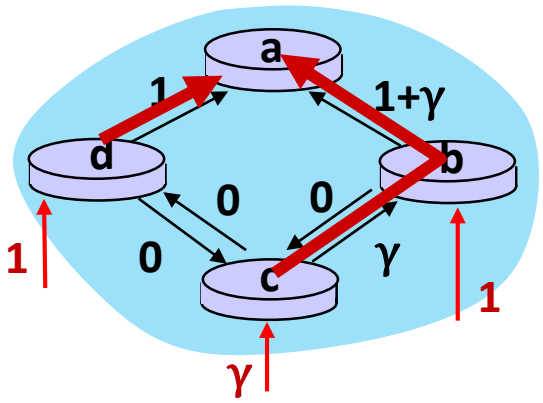
- 출발지를 제외한 n개의 노드가 있다고 가정
- n번의 반복 과정 각각에서, N'에 포함되지 않은 모든 노드 w를 확인해야 함
- 전체 비교 횟수: $n(n+1)/2$ → 시간 복잡도 O(n^2)
- 우선순위 큐를 힙(heap) 자료구조를 활용해서 구현하면 O($nlogn$)으로 개선 가능

메시지 복잡도:

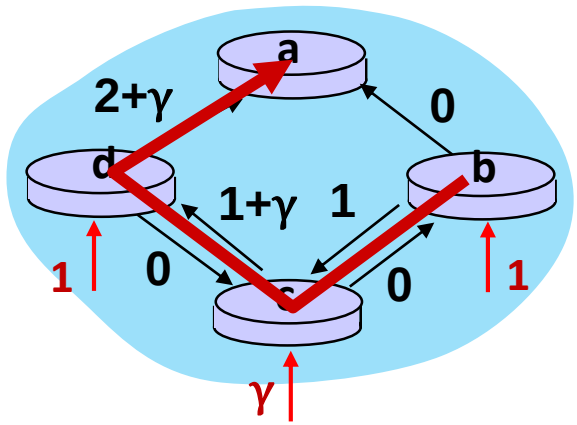
- 각 라우터는 자신의 링크 상태 정보를 다른 n개의 라우터에게 전파해야 함
- 효율적인 플러딩 알고리즘을 사용하면, 하나의 출발지에서 보낸 링크 상태 정보가 O(n)개의 링크를 지나 전체 네트워크에 전파될 수 있음
  - [예] 각 라우터는 순서번호로 최신 링크 상태 정보만 식별하고, 중복 정보는 폐기
- 각 라우터의 메시지가 O(n)개의 링크를 지나므로, 전체 메시지 복잡도는 O(n^2)

Dijkstra's algorithm: oscillations possible

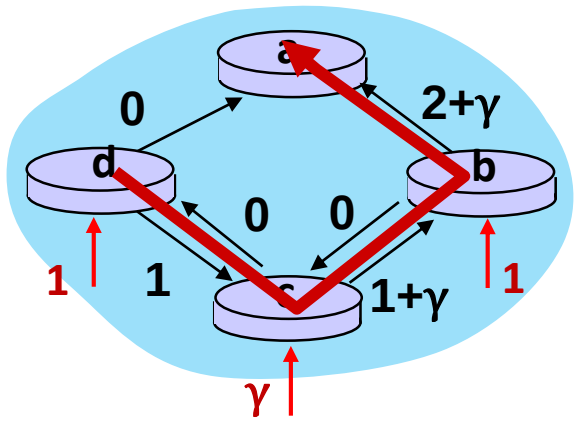
- 링크 비용이 트래픽에 따라 달라지는 경우, 경로 진동 (route oscillations) 현상이 발생할 수 있음
- 경로 진동의 간단한 시나리오:
  - routing to destination '*a*', traffic entering at '*b*', '*c*', '*d*' with rates 1, γ (<1), 1
  - link costs are directional, and volume-dependent



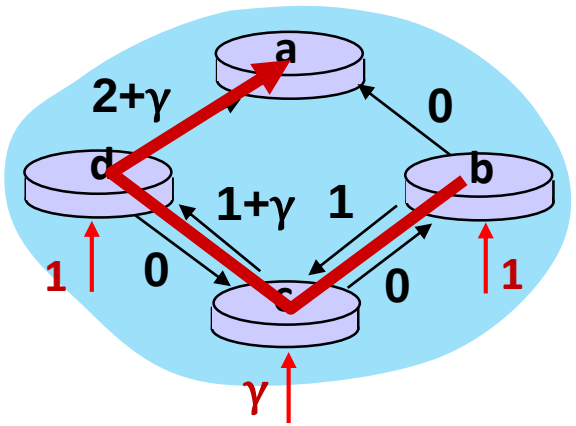
최초 라우팅



b와 c가 a로의 더 나은 경로를 감지함 (시계방향)



b, c, d가 a로의 더 나은 경로를 감지함 (반시계방향)



b, c, d가 a로의 더 나은 경로를 감지함 (시계방향)

Q & A